

CQRS Pattern, Mediator Implementation, and Azure Functions

Modern Backend Architecture for Scalable Applications



Leonardo Tavárez

CQRS Pattern, Mediator Implementation, and Azure Functions

Modern Backend Architecture for Scalable Applications



P3

Course Name: Advanced Backend Architecture

Instructor: Leonardo Tavárez

Learning Objectives



CRUD Limitations

Understand limitations of traditional CRUD.



CQRS Pattern

Learn the **CQRS** architectural pattern.



Mediator & MediatR

Understand the **Mediator** pattern and MediatR in .NET.



Scalable Azure Functions

Build scalable architectures using **Azure Functions**.



CQRS + Serverless

Combine **CQRS** with **serverless** computing.

Traditional CRUD Architecture



All logic goes through the **same pipeline**



Read and write operations are **mixed**

Problems with Traditional CRUD



Tight coupling
between
components



Services become
**large and
complex**



**Difficult
scalability**

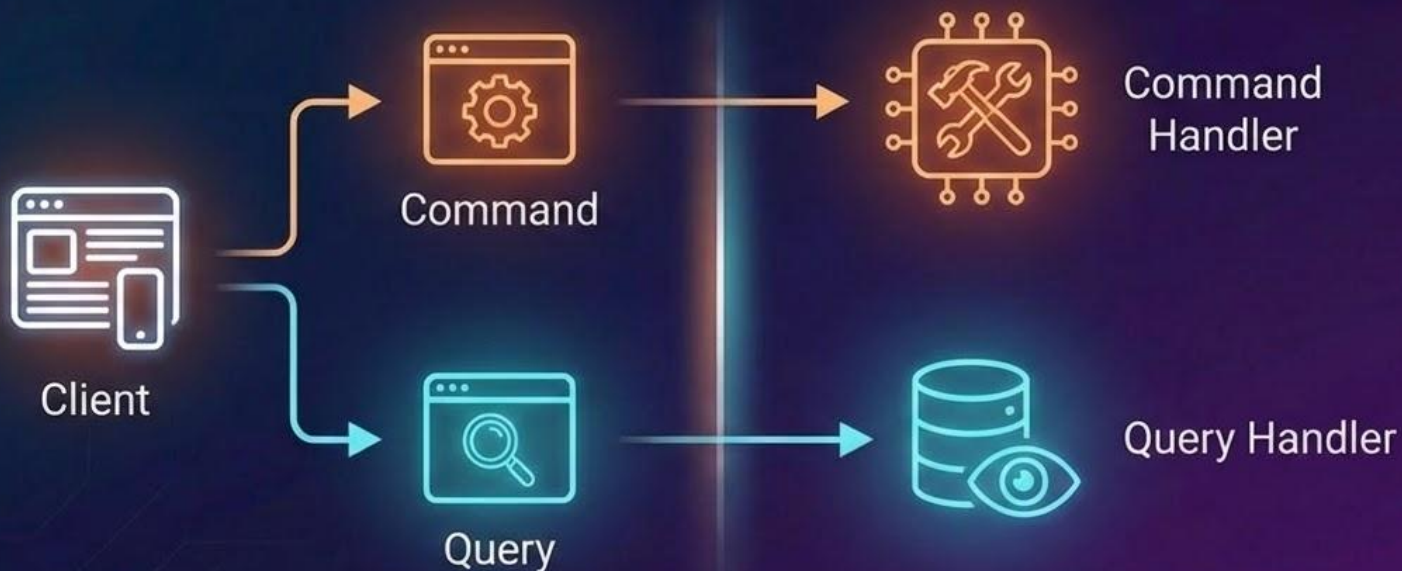


Mixing read and
write
responsibilities

What is CQRS?



CQRS Architecture



Commands vs Queries

Commands



- CreateUser
- UpdateProduct
- DeleteOrder

Modify State & Data



Queries



- GetUserById
- GetOrders
- SearchProducts

Return Data, No Side Effects



Benefits of CQRS



Clear separation
of responsibilities



Improved
scalability



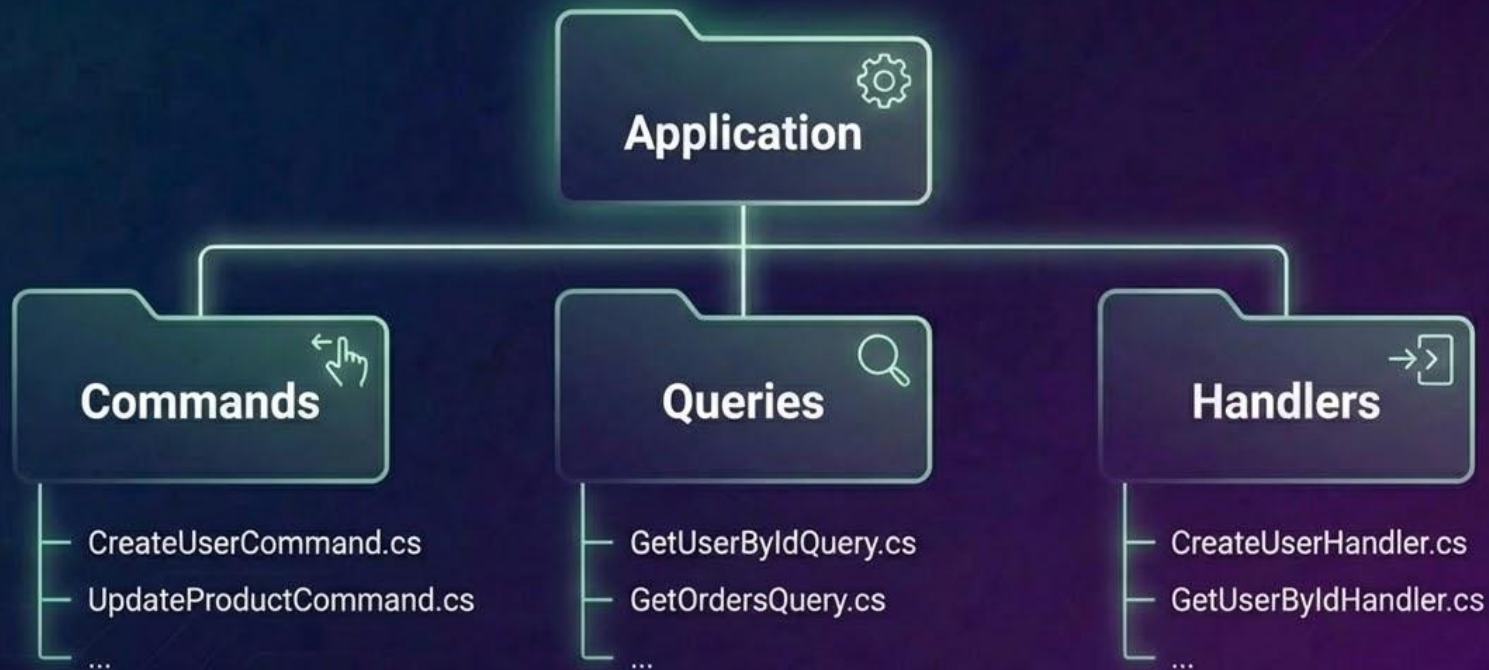
Easier
maintenance



Better performance
for read-heavy
systems



CQRS Project Structure



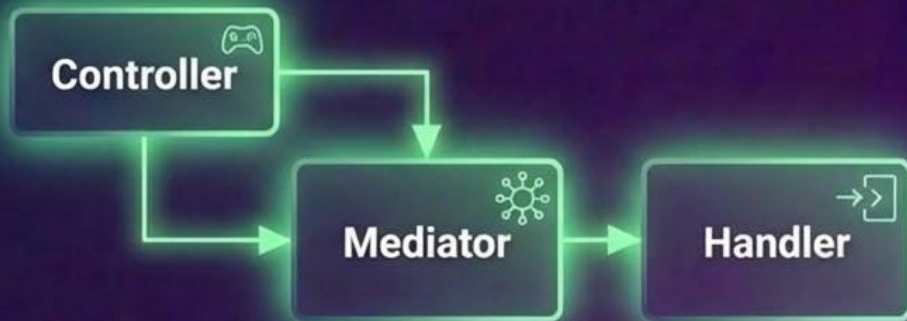
The Mediator Pattern

Mediator centralizes communication
between components.

Instead of:



We use:



Benefits of Mediator



Reduces coupling

Components are loosely connected, promoting modularity.



Simplifies controllers

Controllers become thin, focusing only on request dispatching.



Improves separation of concerns

Isolates business logic from infrastructure and UI.



Organizes application logic

Provides a clear structure for handling commands and queries.

MediatR Library

.NET

MediatR is the most popular mediator implementation in .NET.



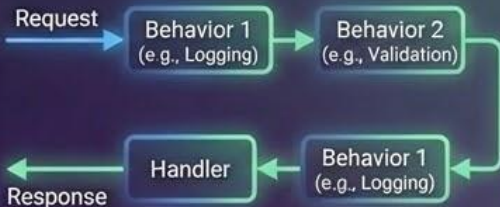
**Request /
Response model**



**Command and
Query handlers**



Pipeline behaviors



```
dotnet add package MediatR
```


Command Example

Command Definition

```
public class CreateUserCommand : IRequest<int>
{
    public string Username { get; set; }
    public string Email { get; set; }
}
```

Command Handler

```
public class CreateUserCommandHandler :
    IRequestHandler<CreateUserCommand, int>
{
    private readonly IUserRepository _repository;

    public CreateUserCommandHandler(IUserRepository repository)
    {
        _repository = repository;
    }

    public async Task<int> Handle(CreateUserCommand request,
        CancellationToken cancellationToken)
    {
        var user = new User(request.Username, request.Email);
        await _repository.AddAsync(user);
        return user.Id;
    }
}
```



How Commands Work

- Represents an **intent to change state**.
- Handled by exactly **one** specific handler.
- Encapsulates the **business logic** for the action.

Query Example

Query Definition

```
public class GetUserByIdQuery : IRequest<UserDto>
{
    public int UserId { get; set; }
}
```

Query Handler

```
public class GetUserByIdQueryHandler :
    IRequestHandler<GetUserByIdQuery, UserDto>
{
    private readonly IReadContext _context;

    public GetUserByIdQueryHandler(IReadContext context)
    {
        _context = context;
    }

    public async Task<UserDto> Handle(GetUserByIdQuery
        request, CancellationToken cancellationToken)
    {
        return await _context.Users.AsNoTracking().
            .FirstOrDefaultAsync(u => u.Id == request.UserId);
    }
}
```



How Queries Retrieve Data

- Focus on **data retrieval** only, no state modification.
- Can target **read-optimized** databases or replicas.
- Completely **separate** from command logic and domain model.

Controller Using Mediator

Controllers send requests to the **Mediator** instead of calling services directly. This promotes loose coupling.



```
[ApiController]
[Route("api/[controller]")]
public class UsersController : ControllerBase
{
    private readonly IMediator _mediator;

    public UsersController(IMediator mediator)
    {
        _mediator = mediator;
    }

    [HttpPost]
    public async Task<IActionResult> CreateUser(CreateUserCommand command)
    {
        var userId = await _mediator.Send(command);
        return Ok(userId);
    }
}
```


What is Azure Functions?



Azure Functions is a **serverless** compute platform.



Event-driven:
Reacts to events
in real-time.



Automatic scaling:
Scales resources up
or down instantly.



Pay-per-use:
Pay only for the time
your code runs.



Cloud native:
Built for the cloud
ecosystem.

Azure Functions Triggers

Azure Functions are **event-driven**, meaning they are executed in response to specific events or **triggers**.



HTTP trigger

Runs when an **HTTP request** is received. Ideal for building APIs.



Timer trigger

Runs on a predefined **schedule**. Useful for background jobs and maintenance tasks.



Queue trigger

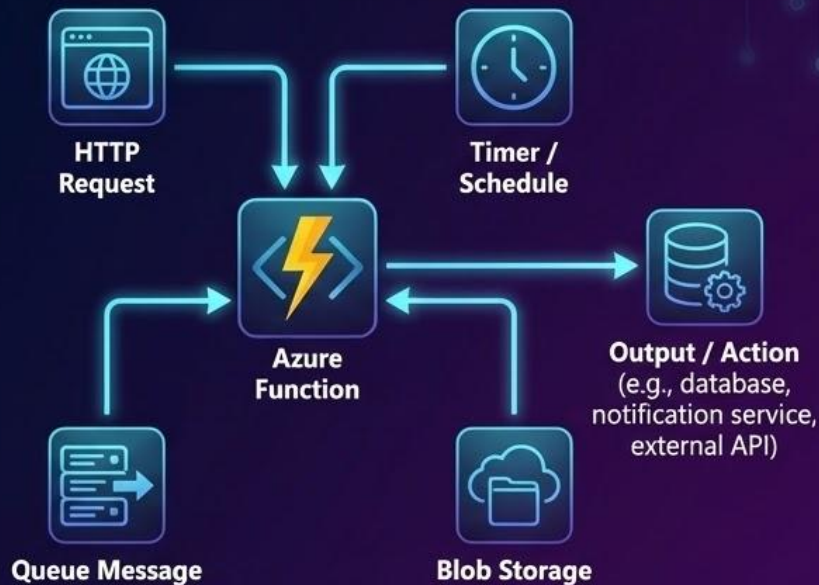
Runs when a new **message** is added to a queue. Enables asynchronous processing.



Blob trigger

Runs when a new or updated **blob** (file) is detected in a container.

Event-Driven Architecture



When Should You Use CQRS?

Use CQRS when:



Applications **grow in complexity**



Read/write workloads differ significantly



Microservices architecture is used

Avoid CQRS when:



Applications are **small** and simple

Key Takeaways



CQRS separates reads and writes.



Mediator decouples application components.



MediatR simplifies CQRS in .NET.



Azure Functions enable scalable serverless backends.